Airport Management System

Software Design Patterns Documentation

Prepared by: *Ajla Alcani*

Document Overview

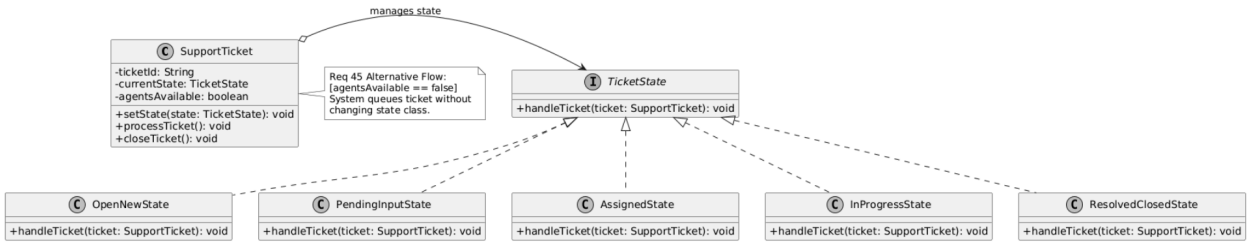
This document presents the implementation and structural mapping of five foundational Software Design Patterns within the Airport Management System. It expands upon our previous behavioral and interaction models by formally defining the static class architectures used to solve specific structural, behavioral, and creational problems across multiple system modules.

Objective

The primary goal of this report is to justify these architectural choices based on established software engineering principles, specifically the Single Responsibility and Open/Closed Principles. Through strict UML Class Diagrams, this document demonstrates how the system satisfies its core functional requirements while securely managing alternative workflows, handling exceptions gracefully, and preventing tight coupling between independent subsystems.

Design Pattern 1: State Pattern

- **Requirement Mapped:** Req 45 (Track service requests).
- **Problem:** A support ticket has a strict lifecycle: Open, Pending, Assigned, In Progress, and Resolved. Managing these transitions using large if/else conditional blocks within the SupportController would make the code fragile and prone to illegal state transitions.
- **Solution:** We applied the **State Pattern**. The SupportTicket object delegates its state-specific behavior to dedicated classes that implement the TicketState interface.
- **Justification:** This pattern simplifies the code of the context by eliminating bulky state machine conditionals. It provides a secure structure to handle our alternative flows; for example, the OpenState class specifically enforces the [if no agent online] queueTicket() guard condition.



Airport Management System

Software Design Patterns Documentation

Prepared by: *Eneida Balliu*

Document Objective

The objective of this section is to identify, specify, and justify the architectural design patterns incorporated into the Airport Management System. While our previous business process models (BPMN) and use case narratives outlined operational requirements, this section transitions those requirements into decoupled, scalable, and reusable object-oriented code structures. Implementing these industry-standard design patterns directly addresses our non-functional requirements—specifically system security compliance, audit traceability, and overall architectural extensibility.

Document Overview

This technical specification details a selected suite of 4–5 core software design patterns tailored to optimize different sub-systems within the airport platform. Each design pattern presentation includes:

- **Pattern Classification & Identification:** Defining the structural, creational, or behavioral nature of the selected pattern.
- **Contextual Project Justification:** A formal description explaining exactly which administration use cases (system maintenance or reporting modules) benefit from the pattern's application.
- **Structural Class Diagrams:** UML-compliant class models detailing interfaces, concrete execution implementations, invoker responsibilities, and data delegation logic.

Design Pattern 2: Command Pattern (System Administration Operations)

Definition: The Command Design Pattern is a behavioral pattern that encapsulates a request or operation as a standalone object. This decouples the interface object triggering the action from the service layer components that contain the actual execution logic.

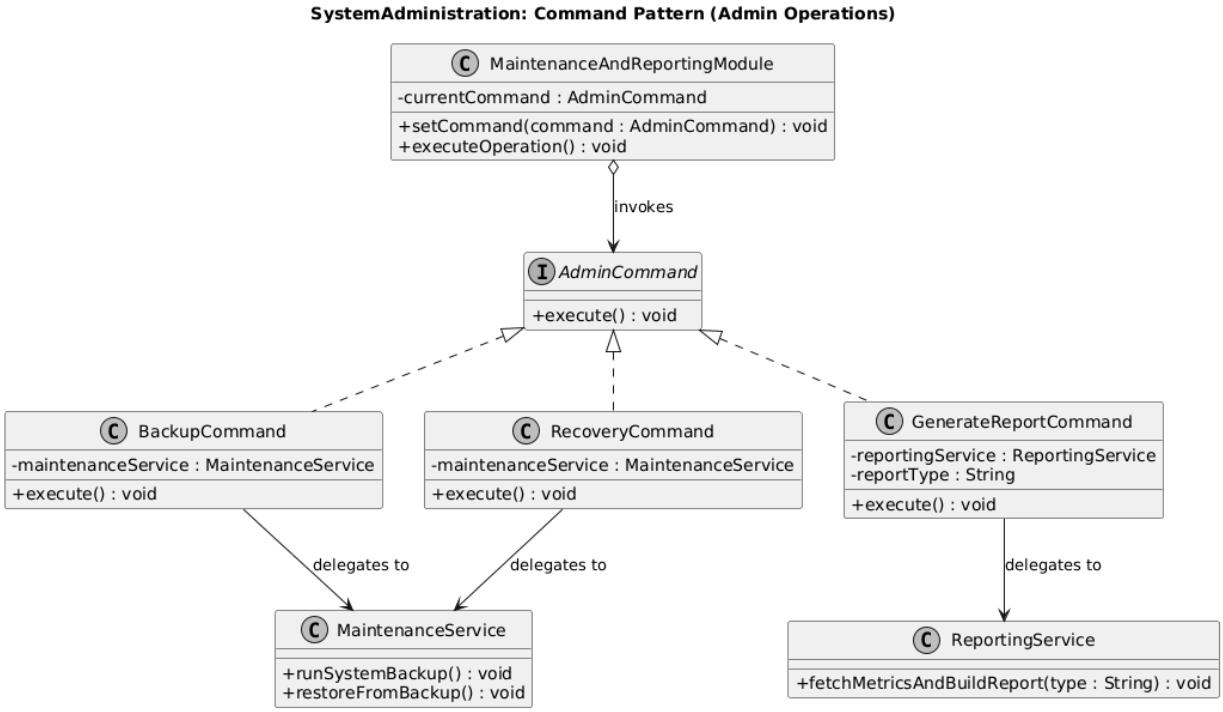
Key Components:

- **The Invoker (MaintenanceAndReportingModule):** The user interface trigger that calls execute() on a command without knowing its underlying database logic.
- **The Command Interface (AdminCommand):** Defines the universal execution contract (execute()).
- **Concrete Commands (BackupCommand, GenerateReportCommand):** Implement execute() by packaging necessary parameters and routing the request to the correct system service.

- **The Receivers (MaintenanceService, ReportingService):** Background domain services containing the actual business logic to run backups, perform data recoveries, or gather system statistics.

Architectural Benefits:

1. **Extensibility:** New administration features can be added as new concrete command classes without modifying existing user interface or controller code.
2. **Graceful Cancellation:** Converting requests into objects makes it straightforward to intercept, halt, or cancel tasks safely before changes alter database states.
3. **Audit Compliance:** Since every administrative action is a distinct object, commands can be sequentially passed to a log manager to record a precise history within the SystemLog.



Airport Management System

Software Design Patterns Documentation

Prepared by: *Betül Kaan*

Document Overview

This document presents the implementation and structural mapping of five foundational Software Design Patterns within the Airport Management System. It expands upon our previous behavioral and interaction models by formally defining the static class architectures used to solve specific structural, behavioral, and creational problems across multiple system modules.

Objective

The primary goal of this report is to justify these architectural choices based on established software engineering principles, specifically the Single Responsibility and Open/Closed Principles. Through strict UML Class Diagrams, this document demonstrates how the system satisfies its core functional requirements while securely managing alternative workflows, handling exceptions gracefully, and preventing tight coupling between independent subsystems.

Design Pattern 3: Observer Pattern (Flight Notification System)

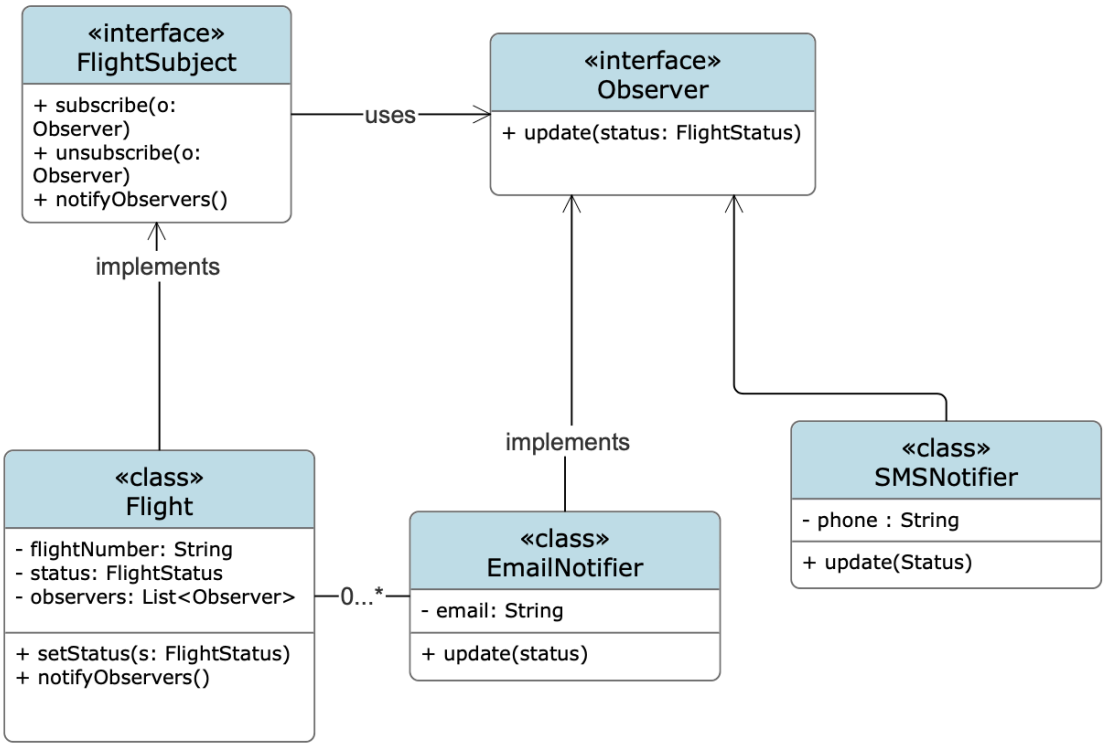
Definition: The Observer Design Pattern is a behavioral pattern that establishes a one-to-many dependency between objects, so that when one object changes state, all its dependents are notified and updated automatically. This decouples the flight data management layer from the notification delivery mechanisms, ensuring that status changes propagate to all registered passengers without the core flight system requiring any knowledge of how those notifications are delivered.

Key Components:

- **The Subject Interface (FlightSubject):** Defines the universal subscription contract subscribe(), unsubscribe(), and notifyObservers() that any observable flight entity must fulfill, allowing observers to register and deregister dynamically at runtime.
- **The Concrete Subject (Flight):** Maintains the authoritative flight state (flight number, current status) alongside an internal list of registered observers. Upon any status change triggered via setStatus(), it automatically invokes notifyObservers() to propagate the update across all subscribers.
- **The Observer Interface (Observer):** Defines the single-method contract update(FlightStatus) that all notification handlers must implement, ensuring the Flight class can communicate uniformly with any type of notifier without coupling to its internal logic.
- **Concrete Observers (EmailNotifier, SMSNotifier):** Implement update() by translating the received FlightStatus into a channel-appropriate alert, a formatted email body or a concise SMS string and delegating delivery to the corresponding service layer (EmailService, SMSService).

Architectural Benefits:

- **Loose Coupling:** The Flight class holds no reference to any specific notifier type. It interacts exclusively through the Observer interface, meaning notification channels can be added, removed, or replaced without modifying any flight management code.
- **Runtime Flexibility:** Passengers can subscribe and unsubscribe dynamically during the application lifecycle, reflecting real-world scenarios such as opting in after booking, changing contact preferences, or automatically unsubscribing after a flight completes.
- **Extensibility:** Introducing a new notification channel, such as PushNotifier or WhatsAppNotifier requires only implementing the Observer interface and registering the new instance; no existing classes are modified, fully satisfying the Open/Closed Principle.


«interface»
FlightSubject
+ subscribe(o:
Observer)
+ unsubscribe(o:
Observer)
+ notifyObservers()
uses
«interface»
Observer
+ update(status: FlightStatus)
implements
implements
«class»
Flight
- flightNumber: String
- status: FlightStatus
- observers: List<Observer>
+ setStatus(s: FlightStatus)
+ notifyObservers()
0...*
«class»
EmailNotifier
- email: String
+ update(status)
«class»
SMSNotifier
- phone : String
+ update(Status)

Airport Management System

Software Design Patterns Documentation

Prepared by: Ani Velo

Module: Security Management Module

Course: Software Modeling and Design

Document Overview

This document presents the implementation and structural mapping of two foundational software design patterns within the **Security Management Module** of the Airport Management System (AMS). By defining static class architectures, this report demonstrates how the security subsystem handles complex conditional screening processes and high-stakes emergency workflows while maintaining loose coupling, single responsibilities, and automated audit trails.

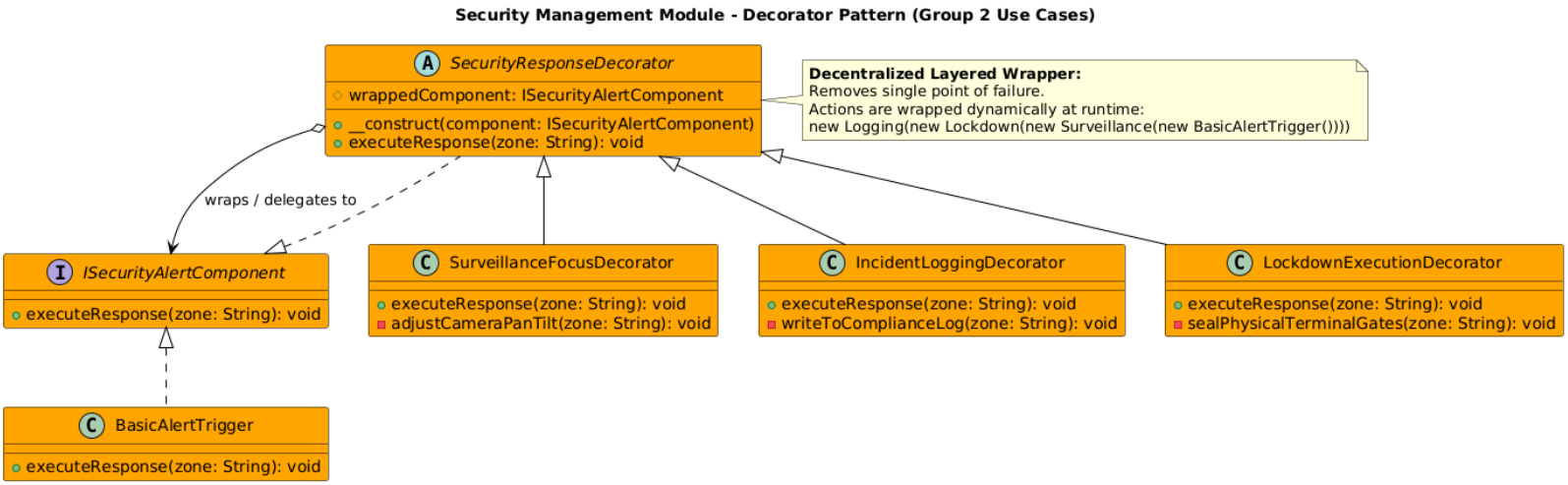
Design Pattern 2: Decorator Pattern (Decentralized Component Wrapper)

- **Requirement Mapped: Requirements Mapped (Group 2):**

1. Security Incident Reporting (UC_SEC_03)
2. Surveillance Monitoring (UC_SEC_04)
3. Emergency Lockdown Procedure (UC_SEC_06)

- **Problem:** When a critical security event is triggered, the system must perform multiple layers of operational safety steps—adjusting camera feeds (UC_04), writing compliance records (UC_03), and sealing physical gates (UC_06). Entrusting this sequence to a single, centralized controller creates a fragile architecture with a dangerous "single point of failure." If that controller component fails, the entire emergency loop crashes. Conversely, hardcoding these functions sequentially makes the system rigid. We need a completely decentralized way to dynamically attach these security actions to an event thread at runtime without causing tight coupling.
- **Solution:** We applied the Structural **Decorator Pattern**. We established a lightweight component interface, ISecurityAlertComponent, which defines a standard operational contract: executeResponse(zone: String). We implement a core BasicAlertTrigger class that performs the foundational system alert propagation. We then create an abstract SecurityResponseDecorator that implements the same interface and holds a structural reference to another component instance. Concrete decorator wrappers—SurveillanceFocusDecorator, IncidentLoggingDecorator, and LockdownExecutionDecorator—extend this abstract decorator to add their specialized use case behaviors before or after delegating to the wrapped component instance.
- **Justification:** This architecture completely eliminates any single point of failure. Because every security behavior is isolated inside its own independent wrapper class, the behaviors are linked

together modularly at runtime (e.g., wrapping a lockdown inside a logger, inside a camera adjuster). This provides total flexibility: if the airport requires a standard report without a lockdown, the system simply omits the lockdown decorator wrapper. It satisfies the **Open/Closed Principle**, allowing developers to create and attach new security wrappers (such as an automated local police dispatch alert) without altering any existing codebase.



Airport Management System

Software Design Patterns Documentation

Prepared by: *Alvi Elezi*

Document Objective

The objective of this section is to identify, specify, and justify the architectural design patterns incorporated into the Airport Management System. While our previous business process models (BPMN) and use case narratives outlined operational requirements, this section transitions those requirements into decoupled, scalable, and reusable object-oriented code structures. Implementing these industry-standard design patterns directly addresses our non-functional requirements, specifically system security compliance, audit traceability, and overall architectural extensibility.

Document Overview

This technical specification details a selected suite of 4–5 core software design patterns tailored to optimize different sub-systems within the airport platform. Each design pattern presentation includes:

- **Pattern Classification & Identification:** Defining the structural, creational, or behavioral nature of the selected pattern.
- **Contextual Project Justification:** A formal description explaining exactly which administration use cases (system maintenance or reporting modules) benefit from the pattern's application.
- **Structural Class Diagrams:** UML-compliant class models detailing interfaces, concrete execution implementations, invoker responsibilities, and data delegation logic.

Design Pattern 5: Strategy Pattern (Flight Operations Module)

Definition: The Strategy Design Pattern is a behavioral software design pattern that defines a family of algorithms, encapsulates each one in its own class, and makes them interchangeable at runtime. This allows for switching an object's behavior without changing the code that uses it.

Requirement Mapped: FO_02 Manage Flight Delays

Key Components:

1. Context (FlightController)

The FlightController acts as the context class responsible for selecting and executing the appropriate delay-handling strategy. It does not implement the operational logic itself; instead, it delegates the behavior to a selected strategy object at runtime.

2. Strategy Interface (DelayStrategy)

The DelayStrategy interface defines the common operational method: handleDelay(). All delay-management strategies must implement this interface, ensuring consistency across different operational procedures.

3. Concrete Strategies

These classes implement specialized operational behavior for different delay scenarios: WeatherDelayStrategy, TechnicalDelayStrategy, SecurityDelayStrategy and CrewDelayStrategy. Each strategy contains unique processing logic, scheduling adjustments, and notification procedures.

Architectural Benefits:

1. Better Separation of Concerns

Each strategy encapsulates a specific operational algorithm independently from the controller and other strategies. This improves modularity and keeps responsibilities clearly separated.

2. Improved Scalability

New delay-handling procedures can be introduced without modifying existing system logic. For example, adding an EmergencyDelayStrategy only requires creating a new strategy implementation.

3. Enhanced Maintainability

Operational rules for each delay category are isolated within their respective classes, making updates, debugging, and testing significantly easier.

Class Diagram: